Angelo Kelly

matrixMain.cpp

Project 2 (Matrix Operations)

purpose:

The purpose of this program is to perform manipulation with matrices of square size, focusing on 2x2 and 3x3 matrices, characterize by the lack of user input, as it runs on 2D arrays of fixed sizes.

Inputs:

The code of the project does not require user input for its functionality, as it uses fixed matrices to determine the sizes and the parameters with which the functions will run.

Outputs:

Here is a list of the outputs the program will produce:

- A message stating the number of rows and columns of the matrix. (This comes from the bottom comment section where the sizeof() function was explained. Since we were instructed to not use it for the project, I decided to not modify anything from that section specifically.)
- Starting messages each function as soon as they start.
- Display 2D array selected.
- Result of addition of matrices
- Shows the transpose of the matrix.
- Outputs the determinant of 2x2 and 3x3 matrices
- Finds a target value in the matrix and shows its location in the matrix.
- Showcase the result of a matrix multiplication.
- Ending messages for each function as soon as they are done.
- Warning messages if matrices sizes are invalid.

Variables

In order to make this program work as desired, the following variables were used:

Variable	Type	Purpose
matrix_local[ ][3]	int array	Parameter for functions to only accept 2D arrays with 3 columns.
matrix_local_rows_total	int	Parameter for functions to know the number of rows in the array.
matrix_one	int array	Function's parameter establishes which array should be read first before the operation is performed.
matrix_one_rows_total	int	Parameter for functions to know the number of rows in the array.
matrix_two	int array	Establishes the array which should be read second before doing the operation.
matrix_two_rows_total	int	Stores the number of rows of the second matrix provided as parameter for functions.
sum_result_matrix	int array	Variable created inside the MatrixAddition function to display the result of the sum of two matrices.
size	int	Parameter for functions to know the size of square matrices.
determinant	int	Depending on the size amount, this variable stores operations to find the determinant of matrices before being displayed by the function.
rows	int	Used in the SearchValue function to know the size of matrix as parameter.
target	int	Used in the SearchValue function, intended to be the specific value the user wishes to find in the matrix-array.

value_found	bool	Boolean operator to determine if the value was found or not.
matrix_a	int array	First matrix to be chosen as parameter for the MatrixMultiplication function.
matrix_b	int array	Second matrix to be chosen as parameter for the MatrixMultiplication function.
rows_a	int	Number of rows of matrix_a, declared as parameter for the multiplication function.
cols_a	int	Number of columns of matrix_a, declared as parameter for the multiplication function
rows_b	int	Same idea as rows_a but designed as parameter for matrix_b in the multiplication function.
cols_b	int	Same idea as cols_a but designed as parameter for matrix_b in the multiplication function.
multiplication_matrix_result	int array	Array variable designed inside the MatrixMultiplication function to store and later display the result of the matrix multiplication.
testing_matrix_one	int array	First testing array that may be used as “user input” and ensure the functionality of the functions.
testing_matrix_two	int array	Similar to the previous one but using different numbers so that functions as SearchValue or Determinant yield different values.
size_row_one	int	Provided variable to know the number of rows in the first matrix tested.

size_col_one	int	Provided variable to know the number of columns in the first testing matrix.

Key Design Choices

Early return: As a simple yet efficient addition, I decided to implement early returns whenever the parameters of the function made matrices operation impossible, thus exiting the function with a warning message. This is the only measure for edge cases and errors since the instructions stated the project will be less focus on those aspects.

determinant variable: This integer is declared in the Determinant function, and it stores the very formulas used to find the determinant of actual matrices of sizes 2x2 and 3x3. Given the fact that matrices beyond those sizes are above the current level of the course, I decided to just assign the formula as values for the variable so that it might be easier for the compiler to read, and for me to write. However, I do recognize that it might be troublesome to read the determinant of a 3x3 size matrix for other software engineers.

The triple nested for-loop: Since matrices multiplication require multiple steps; as it multiplies the rows of a matrix A by the columns of a matrix B, given that every element of said multiplication must be added together and be assigned to their specific cell; a third loop was required. Now, the first two loops can travel between the elements in each row and column of the resulted matrix, while the third loop iterates through each element in the rows and columns of matrix A and B to perform the required operations before storing the results in their cells.

Program steps (Algorithm)

1- Reads that the parameters of each function have been established properly.
2- If the input invalidates the rules for matrix operations, the function terminates with a warning message.
3- Otherwise, it will display the result of each function in the following order: It will print the intended matrix array, followed by the sum of two matrices, then the transpose of one matrix, its determinant, a value that might be stored in the matrix (if no, a message is displayed informing the user about it), and finally, a multiplication of matrices.
4- The program stops

Angelo Kelly

Functions

void Print2DArray()
Purpose: Prints the contents of a matrix using square brackets for each row and vertical bars between columns.

void MatrixAddition()
Purpose: Adds two matrices together and prints the resulting matrix.

void TransposeMatrix()
Purpose: Prints the transpose of the matrix, which just switch the position of rows and columns during the loop.

void Determinant()
Purpose: Calculates and prints the determinant of a 2x2 or 3x3 matrix.

void SearchValue()
Purpose: Searches through the matrix for a target value and prints the index where the value is found. If the value is not found, the function lets the user know with a message.

void MatrixMultiplication()
Purpose: Multiplies two matrices and prints the resulting matrix.

int main( )
Purpose: Executes the principal part of the code where the two matrices that will be used for testing are stored.

Sample Input/Output

Output	Input
	testing_matrix_one: [1|2|3] [4|5|6] [7|8|9]
	testing_matrix_two: [12|65|82] [83|2|8] [10|96|67]
row: 3 col: 3 Initializing function: Print2DArray	

[1|2|3] [4|5|6] [7|8|9] Terminating function: Print2DArray... Initializing function: MatrixAddition [13|67|85] [87|7|14] [17|104|76] Terminating function: MatrixAddition... Initializing function: TransposeMatrix [1|4|7] [2|5|8] [3|6|9] Terminating function: TransposeMatrix... Initializing function: Determinant The determinant of the matrix is: 287863 Terminating function: Determinant... Initializing function: SearchValue Target '2' was found at [0],[1] Terminating function: SearchValue... Initializing function: MatrixMultiplication [208|357|299] [523|846|770] [838|1335|1241] Terminating function: MatrixMultiplication...	

Angelo Kelly

Testing

Case 1: A 3x3 matrix passed into void Print2DArray()

Expected Result: The matrix prints with each row inside square brackets and columns separated by vertical bars.

Case 2: Two valid 3x3 matrices passed into void MatrixAddition ()

Expected Result: The function prints a 3x3 result matrix where each value is the sum of the matching values from both input matrices.

Case 3: A 3x3 matrix passed into void TransposeMatrix ()

Expected Result: The function prints a matrix where the original rows and columns are switched.

Case 4: A 2x3 matrix passed into void Determinant ()

Expected Result: The function prints that the determinant cannot be found since this operation can only be performed in square matrices.

Case 5: A matrix and a target value passed into void SearchValue ()

Expected Result: The function prints the index of the target if found, or prints that the target was not found.

Case 6: Two valid matrices passed into void MatrixMultiplication()

Expected Result: The function prints the matrix multiplication result.